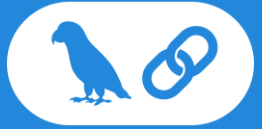


LangChain



- LangChain is a framework for developing applications powered by language models.
- It enables applications that:
 - **Are context-aware:** connect a language model to sources of context (prompt instructions, few shot examples, content to ground its response in, etc.)
 - **Reason:** rely on a language model to reason (about how to answer based on provided context, what actions to take, etc.)



Why LangChain?



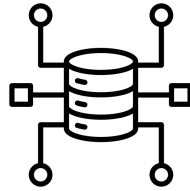
- **Components**
 - LangChain makes it easy to swap out abstractions and components necessary to work with language models.
- **Customized Chains**
 - LangChain provides out of the box support for using and customizing 'chains' - a series of actions strung together.
- **Speed**
 - This team ships insanely fast. You'll be up to date with the latest LLM features.
- **Community**
 - Wonderful discord and community support, meet ups, hackathons, etc.

LangChain Components



LangChain

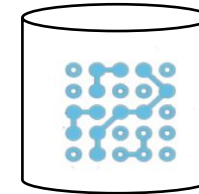
Components



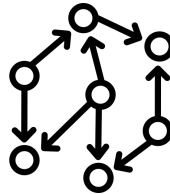
Schema



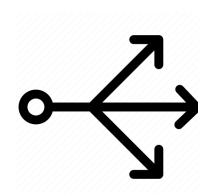
Indexes



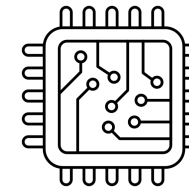
Vector Stores



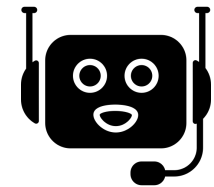
Models



Text Splitters



Memory



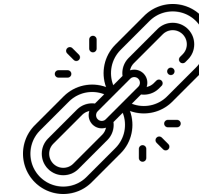
Agents



Prompts



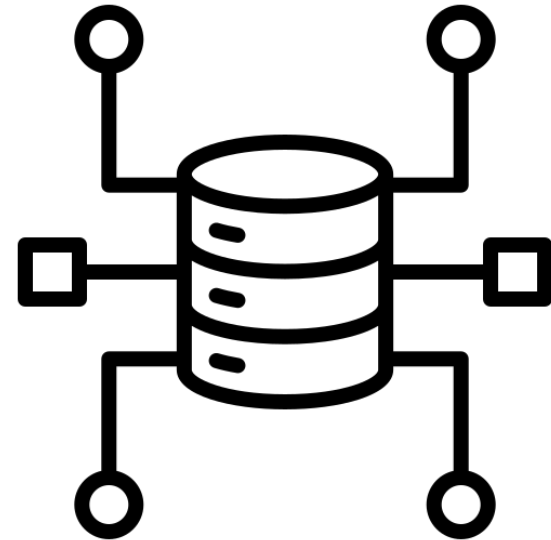
Retrievers



Chains



- A “**Schema**” refers to structured definitions for different types of data and messages that are used to interact with Large Language Models (LLMs).
- These schemas help in organizing the data, providing consistency, and ensuring that the interactions with LLMs are well-defined and structured.



Why Do We Need Schema?



- Consistency
- Validation
- Interoperability
- Scalability
- Ease of Use

Key Components of Schema



- **Text:** A collection of characters, words, sentences, or paragraphs that form a meaningful segment of natural language. It can include letters, numbers, punctuation marks, and other symbols.

Key Components of Schema



- **Messages:** These are the fundamental building blocks for interacting with LLMs in a conversational manner. Messages are categorized into different types:
 - **SystemMessage:** Provides background context and instructions for the AI.
 - **HumanMessage:** Represents the user inputs or queries.
 - **AIMessage:** Represents the responses generated by the AI.

Key Components of Schema

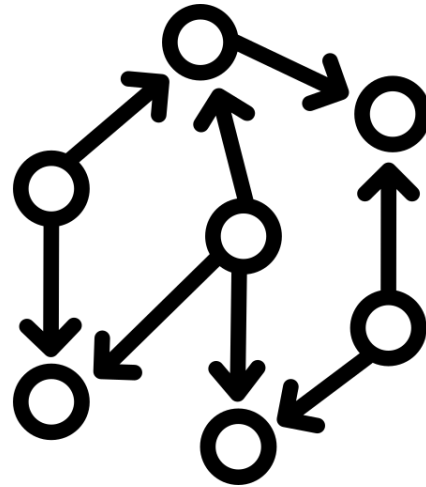


- **Documents:** These are objects that hold pieces of text along with associated metadata. Documents can be used to manage and process text data effectively.
 - **page_content:** The actual text content of the document.
 - **metadata:** Additional information about the document, such as its source, creation time, and any relevant identifiers.

Demo



- Models, in the context of machine learning and artificial intelligence, are mathematical representations or algorithms that have been trained on data to perform specific tasks. These tasks can include predicting outcomes, classifying data, generating text, recognizing patterns, and more.



Why Do We Need Models?



- Automation and Efficiency
- Prediction and Decision Making
- Personalization
- Scalability
- Consistency and Reliability
- Advanced Capabilities

Types of Models



- **Language Models:** These models are designed to generate or comprehend text. They can take text input and produce text output, simulating human-like text generation. Examples include OpenAI's GPT (Generative Pre-trained Transformer) models.
- **Chat Models:** A subset of language models specifically tuned for conversational contexts. They handle a series of messages and can maintain context over a conversation, making them suitable for chatbots and virtual assistants.
- **Function Calling Models:** These models are fine-tuned to provide structured data outputs, which can be particularly useful for tasks like making API calls or extracting specific information in a structured format.
- **Text Embedding Models:** These models convert text into numerical vectors that represent the semantic meaning of the text. They are used in tasks such as text similarity, clustering, and search.

Demo

Prompt Template



- A prompt template is a structured framework that allows for dynamic generation of prompts based on predefined patterns and placeholders. It typically includes fixed text and variables that can be filled with specific values at runtime. Prompt templates are useful when generating multiple prompts with similar structures but varying content or style.

Key Components of a Prompt Template



- **Template String:** The base text with placeholders for variables.
- **Variables:** Parameters that can be substituted into the template to create specific instances of the prompt.
- **Formatting Logic:** Rules or methods for inserting variables into the template.

Demo

Output Parsers



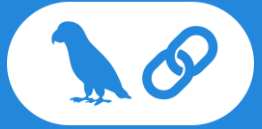
- Output parsers are tools or methods used to extract structured data or information from unstructured or semi-structured textual outputs generated by language models or other natural language processing (NLP) systems. They are essential in converting raw text responses into structured formats that are easier to analyze, manipulate, and integrate into applications or workflows.

Key Functions of Output Parsers

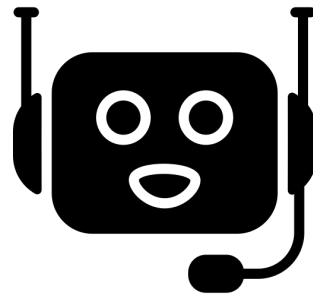


- Structured Data Extraction
- Format Conversion
- Error Handling and Validation
- Schema Definition

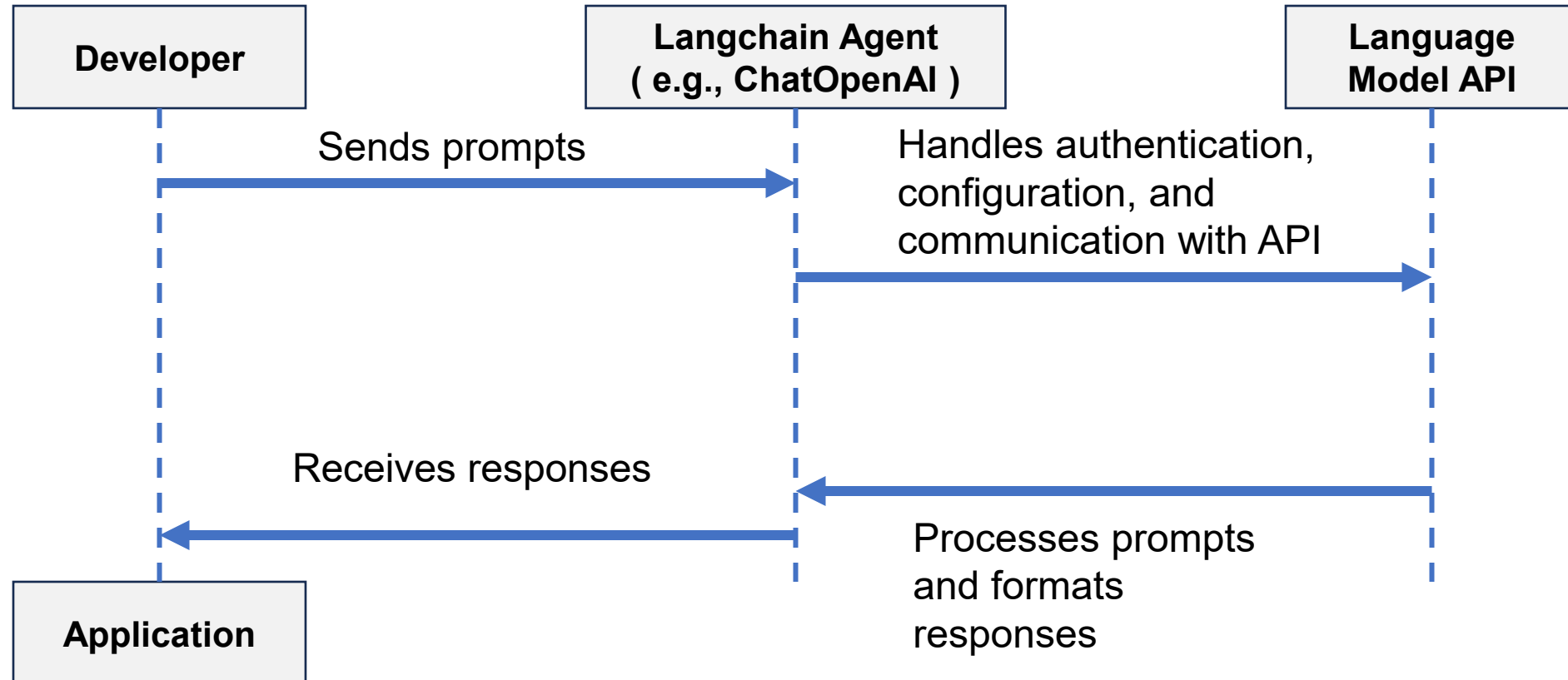
Demo



- They are specialized components within the LangChain framework that act as intelligent assistants, using LLMs to process natural language input and then orchestrating a sequence of actions to achieve a desired outcome.



Agents



Retrieval Augmented Generation (RAG)

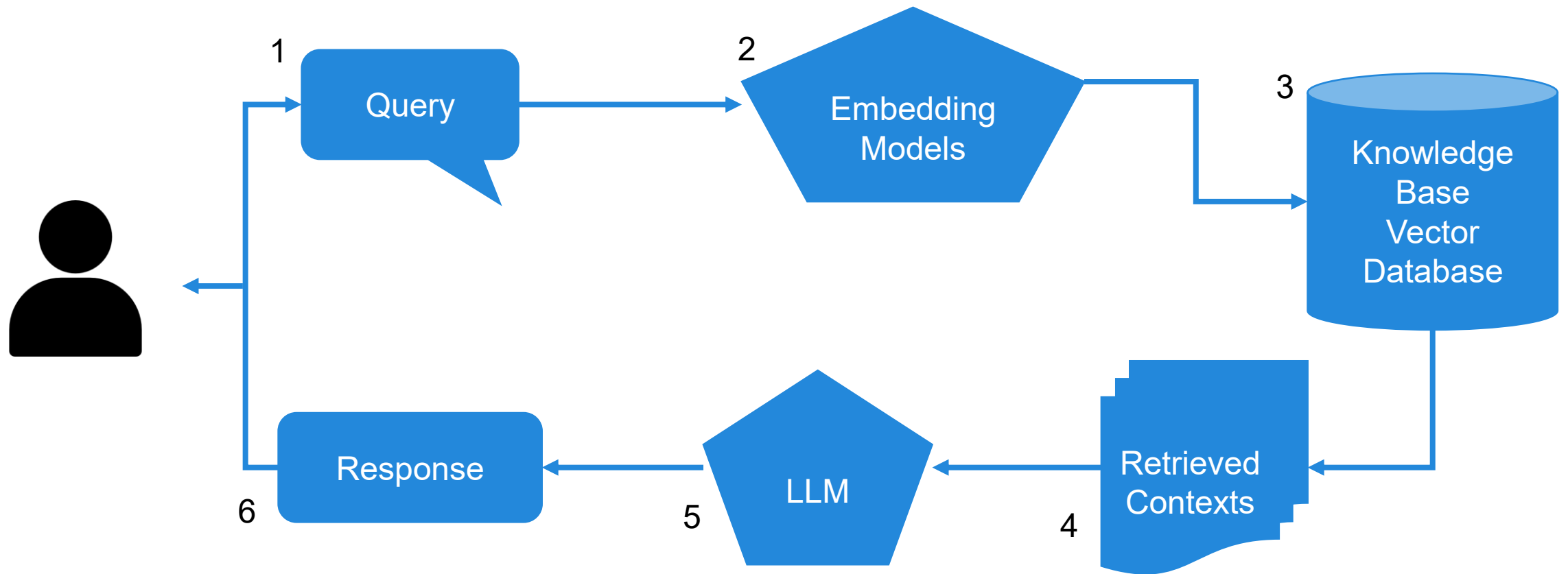


- RAG (Retrieval Augmented Generation) is a technique for augmenting LLM knowledge with additional, often private or real-time, data.
- LLMs are trained on enormous bodies of data but they aren't trained on your data. RAG solves this problem by adding your data to the data LLMs already have access to.

Stages of Retrieval Augmented Generation



Advanced Retrieval Augmented Generation



Retrieval Augmented Generation



Context	-	The context is established via the retrieval process.
Generation	-	Generation is performed by the LLM, which generates the answer by using the retrieved information.

Both elements need to be evaluated separately

And

Together to get an overall score

as well as

Individual scores to pinpoint the aspects to improve

Retrieval Augmented Generation



What is photosynthesis?

RAG model answered, It's the process plants use to convert sunlight into energy.

The correct answer is,

It's the process by which green plants and some other organisms use sunlight to synthesis foods with the help of chlorophyll pigments.

Retrieval Augmented Generation



Ragas framework =

```
{  
  'ragas score': 0.3482,  
  'context  
_relevancy': 0.1296,  
  'faithfulness': 0.8889,  
  'answer_relevancy': 0.9287,  
  'context recall': 0.6370,  
  'harmfulness': 0.0000  
}
```

Retrieval Augmented Generation



Testing Generation Quality

- **Perplexity:** For the language model's fluency
- **BLEU, METEOR, ROUGE:** If there are reference answers, these metrics can help measure the quality of the generated text.
- **Answer Accuracy:** For QA tasks, assess whether the generated answer is correct based on the retrieved documents.

End-to-End Evaluation

- **Exact Match & F1 Score:** In a QA setting, compare the generated answers to reference answers.
- **User Studies:** Have real users rate the quality, relevance, and accuracy of the generated responses.

Retrieval Augmented Generation



Improve the Performance of RAG

- Clean data
- Explore different index types
- Experiment with chunking approach
- Play around with base prompt
- Try meta-data filtering
- Use query routing
- Look into reranking
- Consider query transformations
- Fine-tune embedding model
- Start using LLM dev tools

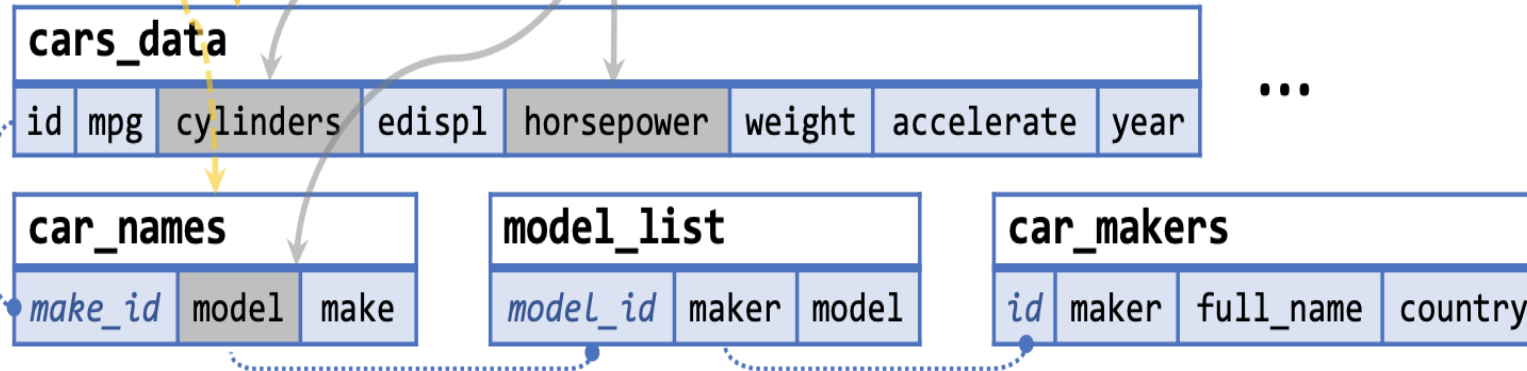
Retrieval Augmented Generation: Text2SQL



Natural Language Question:

For the cars with 4 cylinders, which model has the largest horsepower?

Schema:



Desired SQL:

```
SELECT T1.model
FROM car_names AS T1 JOIN cars_data AS T2
ON T1.make_id = T2.id
WHERE T2.cylinders = 4
ORDER BY T2.horsepower DESC LIMIT 1
```

- Question → Column linking (unknown)
- - - - - Question → Table linking (unknown)
- ... Column → Column foreign keys (known)

Advanced Retrieval Techniques



- Prefetching for Enhanced Performance
 - When to employ prefetching
 - Implementing prefetching: a practical example
- Memoization: A Strategic Optimization Technique
 - Determining the right time for memoization
 - Implementing memoization: a practical illustration
- Concurrent Data Fetching: Enhancing Efficiency in Data Retrieval
 - Determining the optimal use of concurrent fetching
 - Implementing concurrent fetching: a practical PHP example
- Lazy Loading: Enhancing Efficiency in Resource Loading
 - Implementing lazy loading effectively
 - Illustrating lazy loading with React

Demo

Langchian Chains



- Chains are a fundamental concept that allows you to execute complex tasks in a structured and efficient way.
- Chains are invaluable due to their capacity to effortlessly blend diverse components, shaping a singular and coherent application. Through the creation of chains, multiple elements can seamlessly come together.



Why Do We Need Chains?



- Enhanced Complexity Handling
- Modularity and Reusability
- Improved Accuracy and Efficiency
- Flexibility and Scalability
- Contextual Awareness
- Error Handling and Recovery

LLM Chain - The simplest chain



- The LLMChain is a foundational system that includes a PromptTemplate, an OpenAI model (such as a Large Language Model or a ChatModel), and optionally, an output parser. It operates by transforming input parameters using the PromptTemplate into a coherent prompt, which is then fed into the model. The resulting output is further refined and formatted into a usable form by the OutputParser, if provided. This structured approach ensures effective utilization of language models for various applications, enhancing their functionality and utility.



```
prompt = ChatPromptTemplate.from_template(  
    "What is the best name to describe a company  
    that makes {product}?"  
)
```

```
chain = LLMChain(llm=llm,  
    prompt=prompt)
```

SimpleSequentialChain

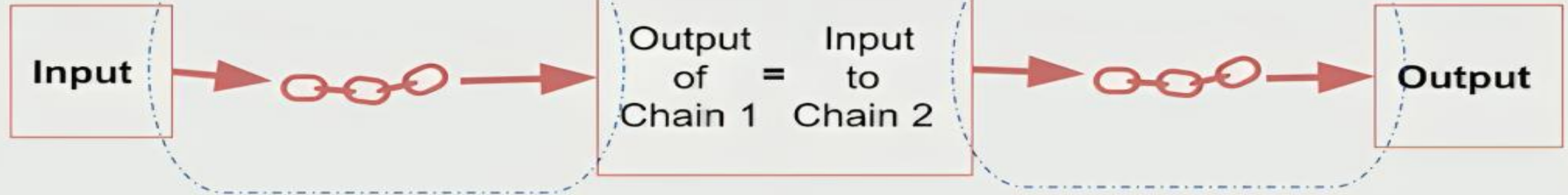


- Simple Sequential Chains allow for a single input to undergo a series of coherent transformations, resulting in a refined output. This sequential approach ensures systematic and efficient handling of data, making it ideal for scenarios where a linear flow of information processing is essential



Chain 1

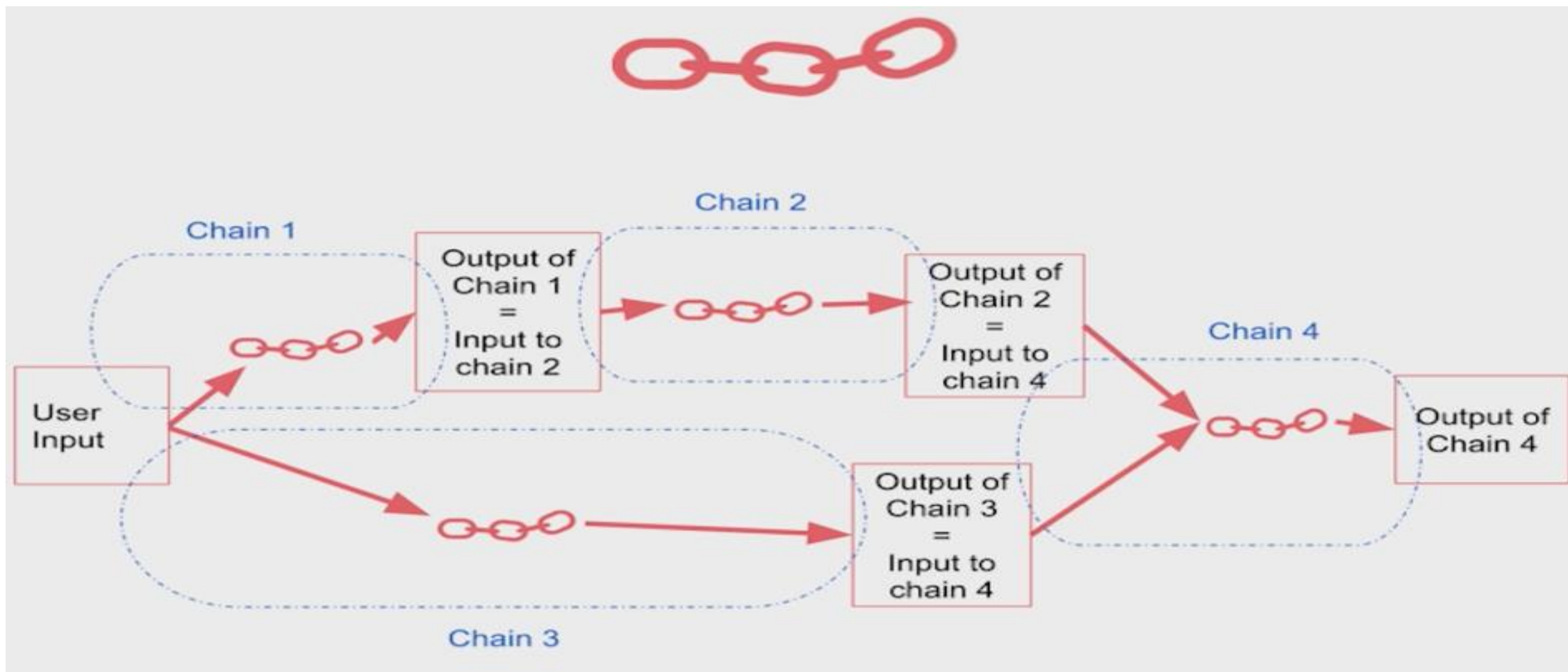
Chain 2



SequentialChain



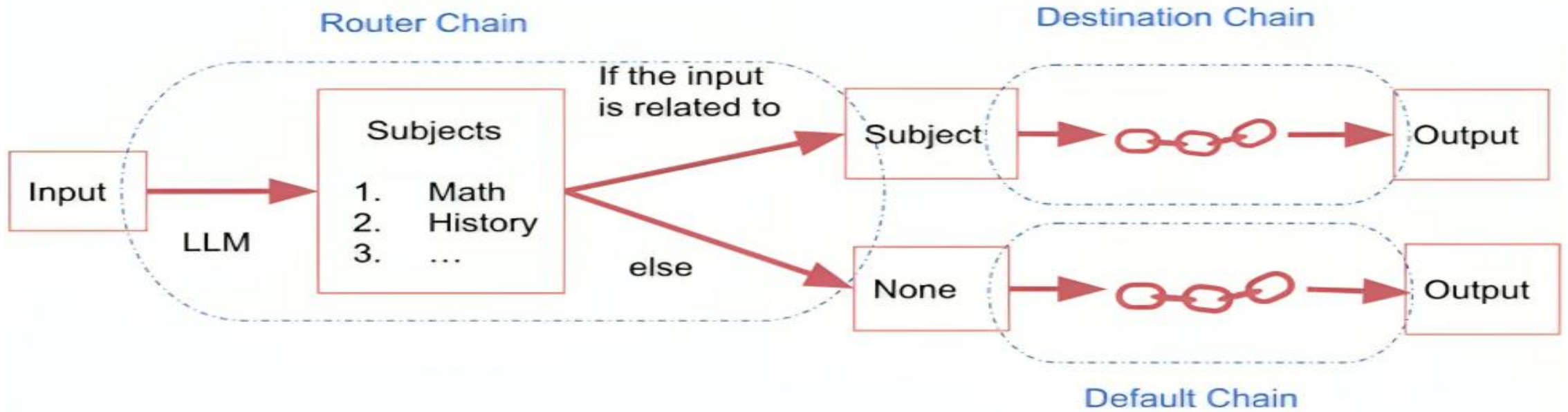
- A sequential chain is a chain that combines various individual chains, where the output of one chain serves as the input for the next in a continuous sequence. It operates by running a series of chains consecutively.



Router Chain



- The Router Chain is used for complicated tasks. If we have multiple subchains, each of which is specialized for a particular type of input, we could have a router chain that decides which subchain to pass the input to.

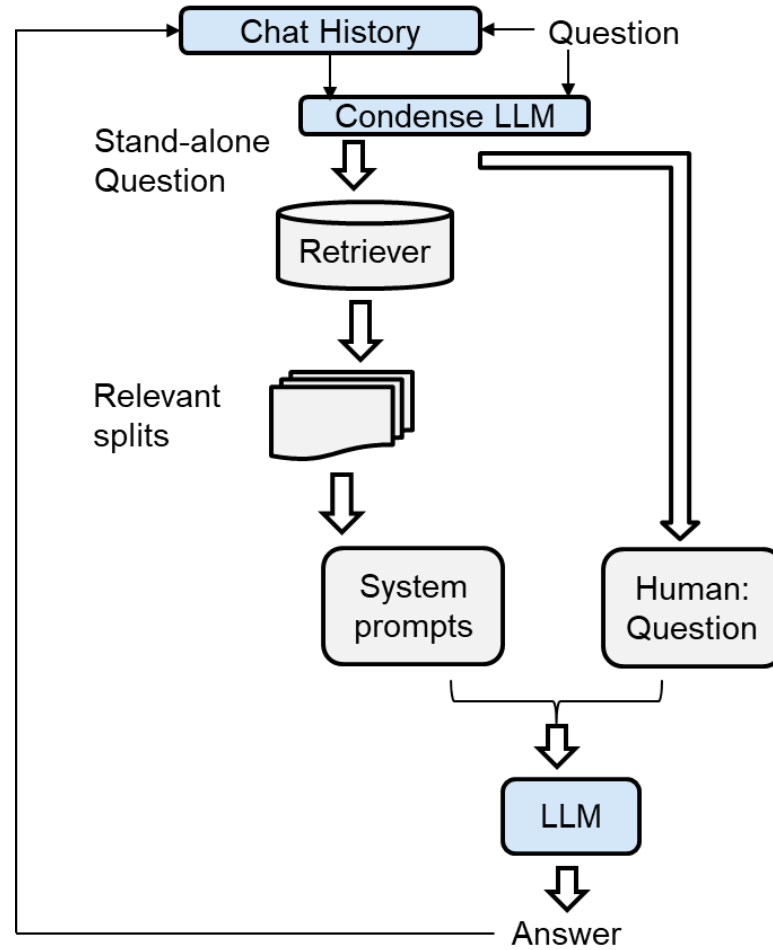


Conversational Retrieval Chain



- The **ConversationalRetrievalChain** is a specialized chain designed to facilitate conversational interactions with a knowledge base, often in the context of **Retrieval Augmented Generation (RAG)**. It combines the ability to retrieve relevant documents with the capacity to maintain conversational context through chat history.

Conversational Retrieval Chain

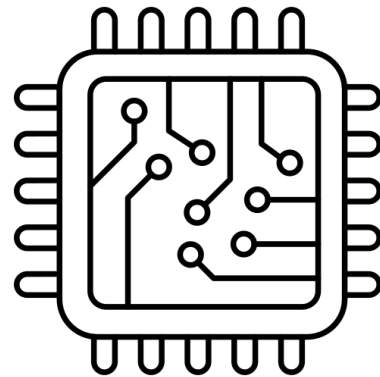


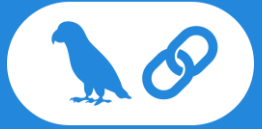
Demo

Memory

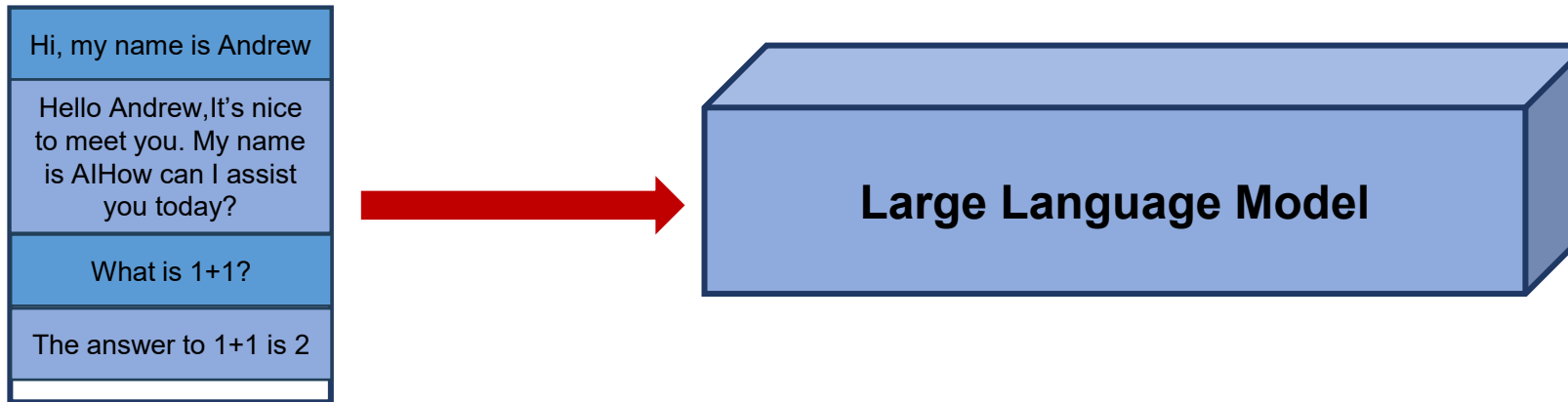


- The memory allows a Large Language Model (LLM) to remember previous interactions with the user. By default, LLMs are stateless meaning each incoming query is processed independently of other interactions.





- Large Language Models are 'stateless'
 - Each transaction is independent
- Chatbots appear to have memory by providing the full conversation as 'context'





ConversationBufferMemory

- This memory allows for storing of messages and then extracts the messages in a variable.

ConversationBufferWindowMemory

- This memory keeps a list of the interactions of the conversation over time. It only uses the last K interactions.



ConversationTokenBufferMemory

- This memory keeps a buffer of recent interactions in memory, and uses token length rather than number of interactions to determine when to flush interactions.

ConversationSummaryMemory

- This memory creates a summary of the conversation over time.

Additional Memory Types



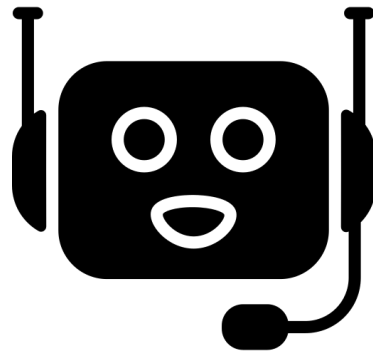
- Vector data memory
 - Stores text (from conversation or elsewhere) in a vector database and retrieves the most relevant blocks of text.
- Entity memories
 - Using an LLM, it remembers details about specific entities.
- You can also use multiple memories at one time. E.g., Conversation memory + Entity memory to recall individuals.
- You can also store the conversation in a conventional database (such as key-value store or SQL)

Demo

Langchian Agents



- Agent is a modular and flexible component that can perform a variety of tasks by leveraging multiple tools and large language models (LLMs). Agents act as intermediaries that can dynamically select and execute actions based on the input they receive. They enable the creation of complex applications that require decision-making and multi-step processes.



Why Do We Need Agents?



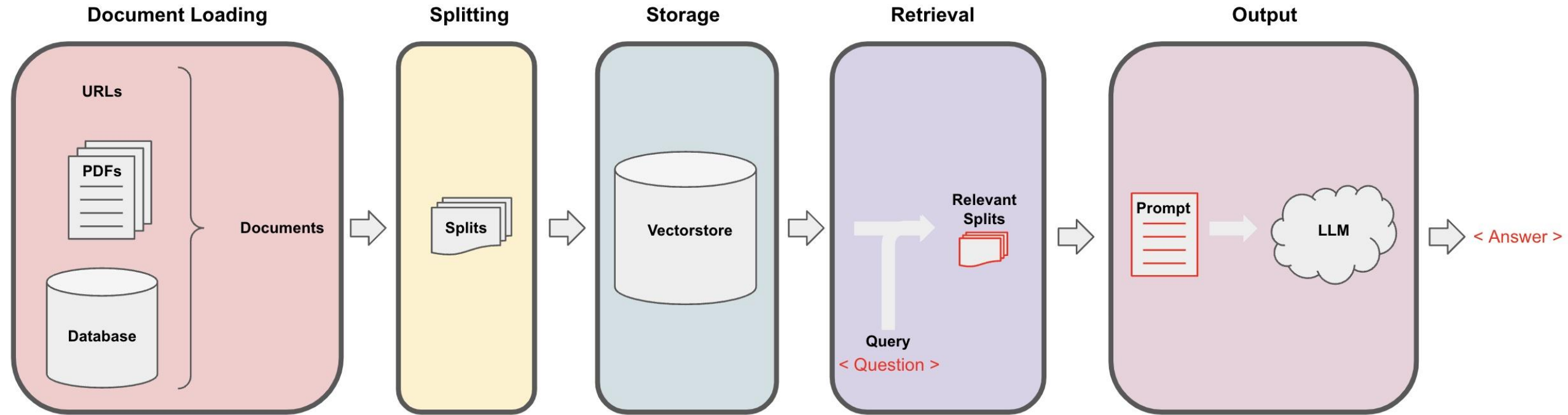
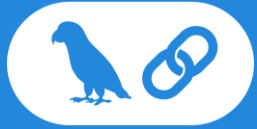
- Automation of Repetitive Tasks
- Consistency in Responses
- Scalability
- Data Processing and Analysis
- Integration with Systems
- Enhanced User Experience
- Cost Efficiency

Retrieval Augmented Generation(RAG)



- In retrieval augmented generation (RAG), an LLM retrieves contextual documents from an external dataset as part of its execution.
- This is useful if we want to ask question about specific documents (e.g., our PDFs, a set of videos, etc).

Retrieval Augmented Generation(RAG)



Document Loading

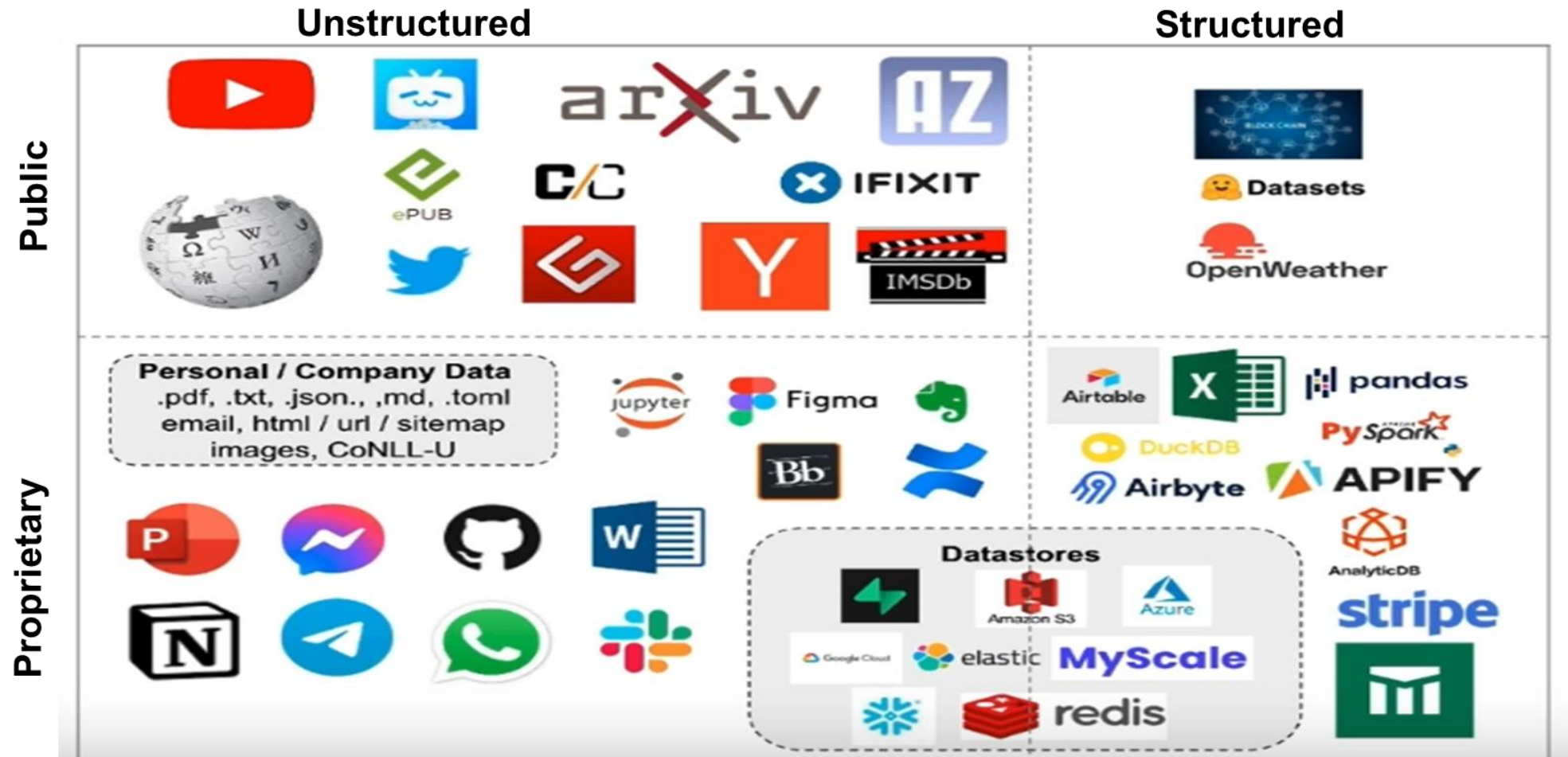
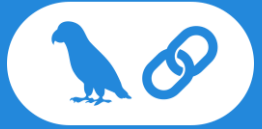


- Document loading refers to the process of fetching and retrieving content from external sources such as websites or databases into an application or system. It is essential for accessing and utilizing data, text, or multimedia resources dynamically, enabling applications to display, process, or manipulate information sourced from various repositories. This capability is crucial for real-time updates, content integration, and efficient data handling in modern software and web development.



- Loaders deal with the specifics of accessing and converting data
 - Accessing
 - Web Sites
 - Data Bases
 - YouTube
 - arXiv
 -
 - Data Types
 - PDF
 - HTML
 - JSON
 - Word, PowerPoint...

Document Loaders

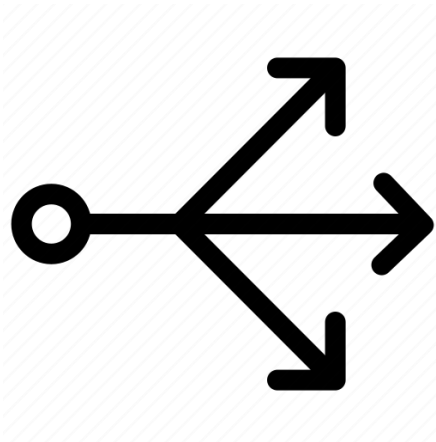


Demo

Document Splitting



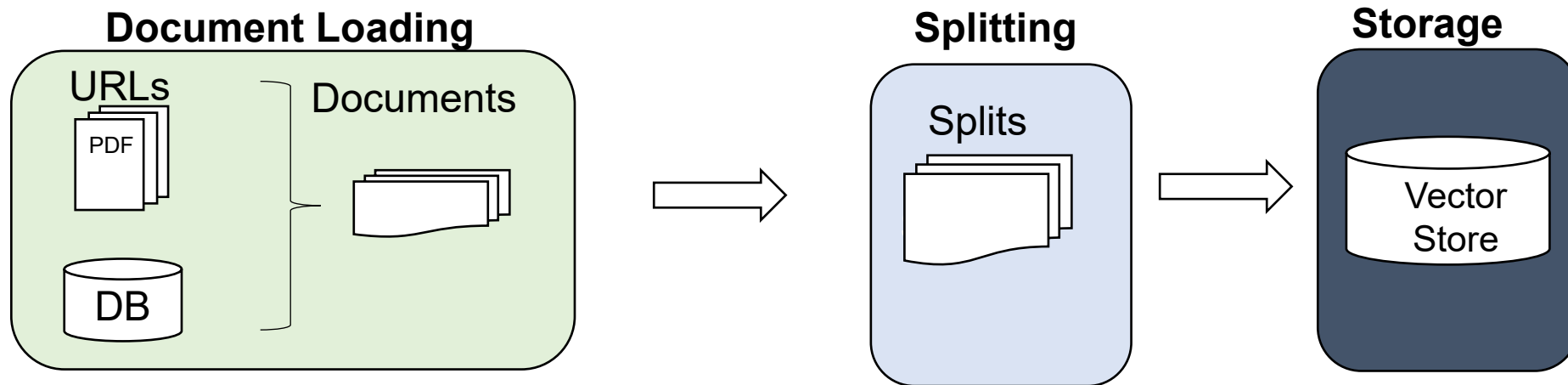
- Document splitting is crucial because it ensures that semantically relevant content is grouped together within the same chunk. This is particularly important when answering questions or performing other tasks that rely on the contextual information present in the documents.



Document Splitting



- Splitting Documents into smaller chunks
- Retaining meaningful relation



Document Splitting



....

on this model. The Toyota Camry has a head-snapping 80 HP and an eight-speed automatic transmission that will

....

Chunk 1: on this model. The Toyota Camry has a head-snapping

Chunk 2: 80 HP and an eight-speed automatic transmission that will

Question: What are the specifications of the Camry?

Example Splitter



```
langchain.text_splitter.CharacterTextSplitter  
(  
    separator: str = "\n\n",  
    chunk_size=4000,  
    chunk_overlap=200,  
    length_function=<builtin function len>,  
)
```

Methods:

- **create_documents()** - Create documents from a list of texts.
- **split_documents()** - Split documents.

Example Splitter



Types of Text Splitters



`langchain.text_splitter.`

- **CharacterTextSplitter()** - Implementation of splitting text that looks at characters.
- **MarkdownHeaderTextSplitter()** - Implementation of splitting markdown files based on specified headers.
- **TokenTextSplitter()** - Implementation of splitting text that looks at tokens.
- **SentenceTransformersTokenTextSplitter()** - Implementation of splitting text that looks at tokens.
- **RecursiveCharacterTextSplitter()** - Implementation of splitting text that looks at characters. Recursively tries to split by different characters to find one that works.

Types of Text Splitters

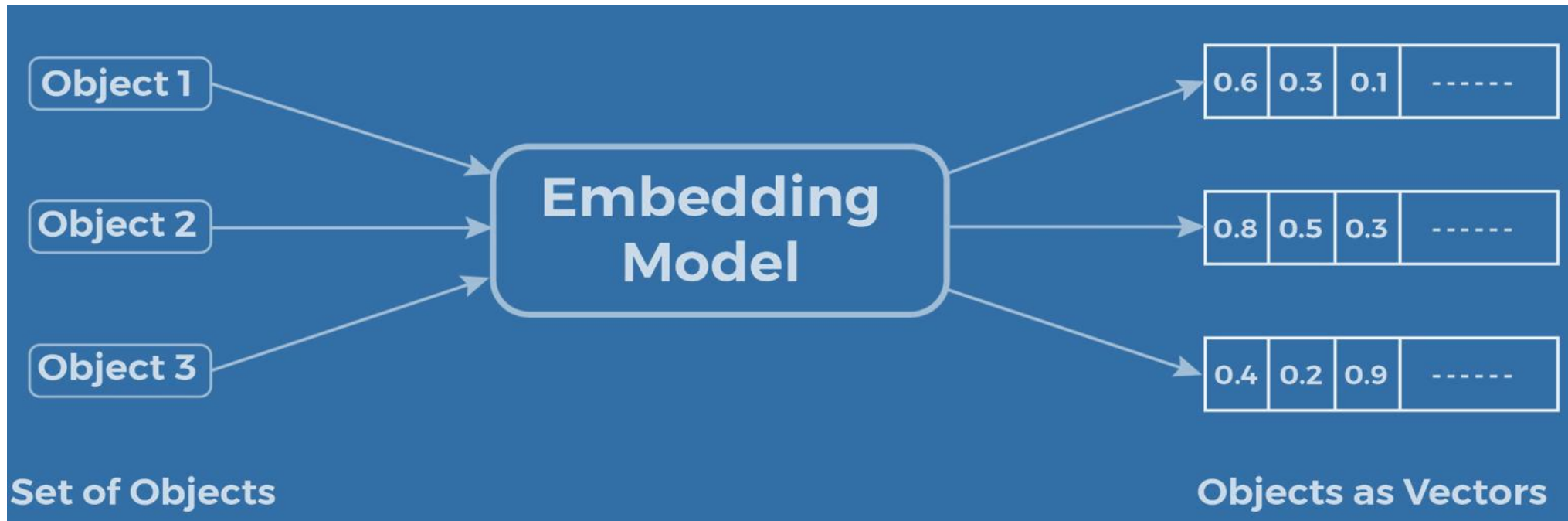
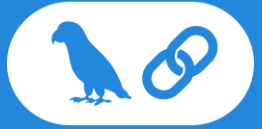


`langchain.text_splitter.`

- **Language()** — for CPP, Python, Ruby, Markdown etc
- **NLTKTextSplitter()** - Implementation of splitting text that looks at sentences using NLTK (Natural Language Tool Kit)
- **SpacyTextSplitter()** - Implementation of splitting text that looks at sentences using Spacy

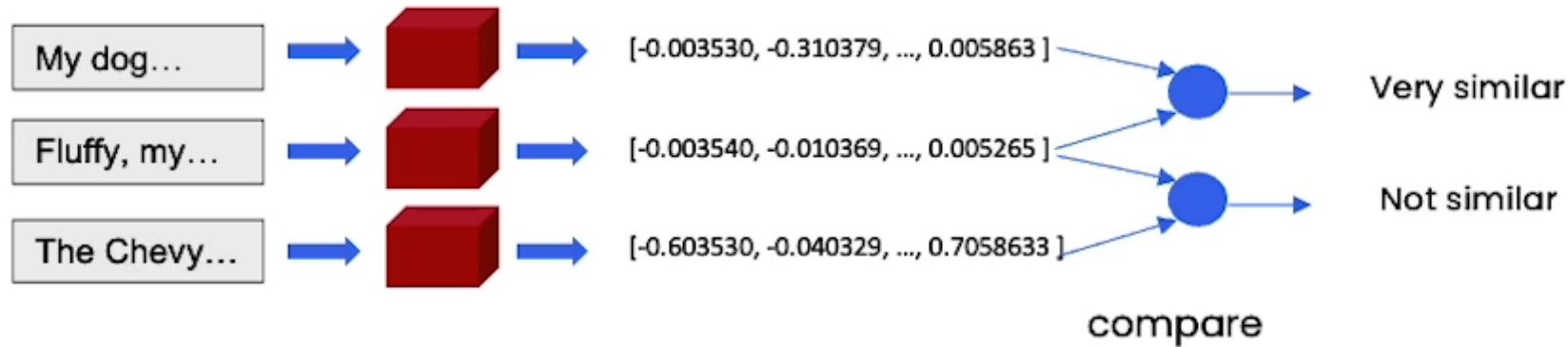
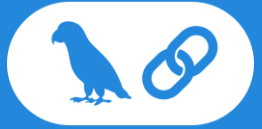
Demo

Embeddings



- Embedding vector captures content/meaning
- Text with similar content will have similar vectors

Embeddings

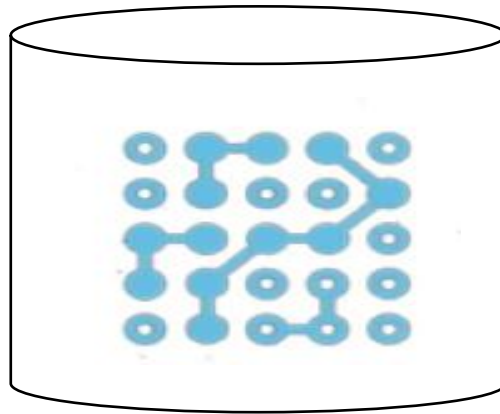


1. My dog Rover likes to chase squirrels.
2. Fluffy, my cat, refuses to eat from a can.
3. The Chevy Bolt accelerates to 60 mph in 6.7 seconds.

Vector Store



- A vector store is a database designed to store and manage numerical vectors, such as embeddings, for efficient retrieval and similarity search. It enables quick and accurate matching of vectors, facilitating tasks like nearest neighbor search, clustering, and recommendation systems based on vector similarity.

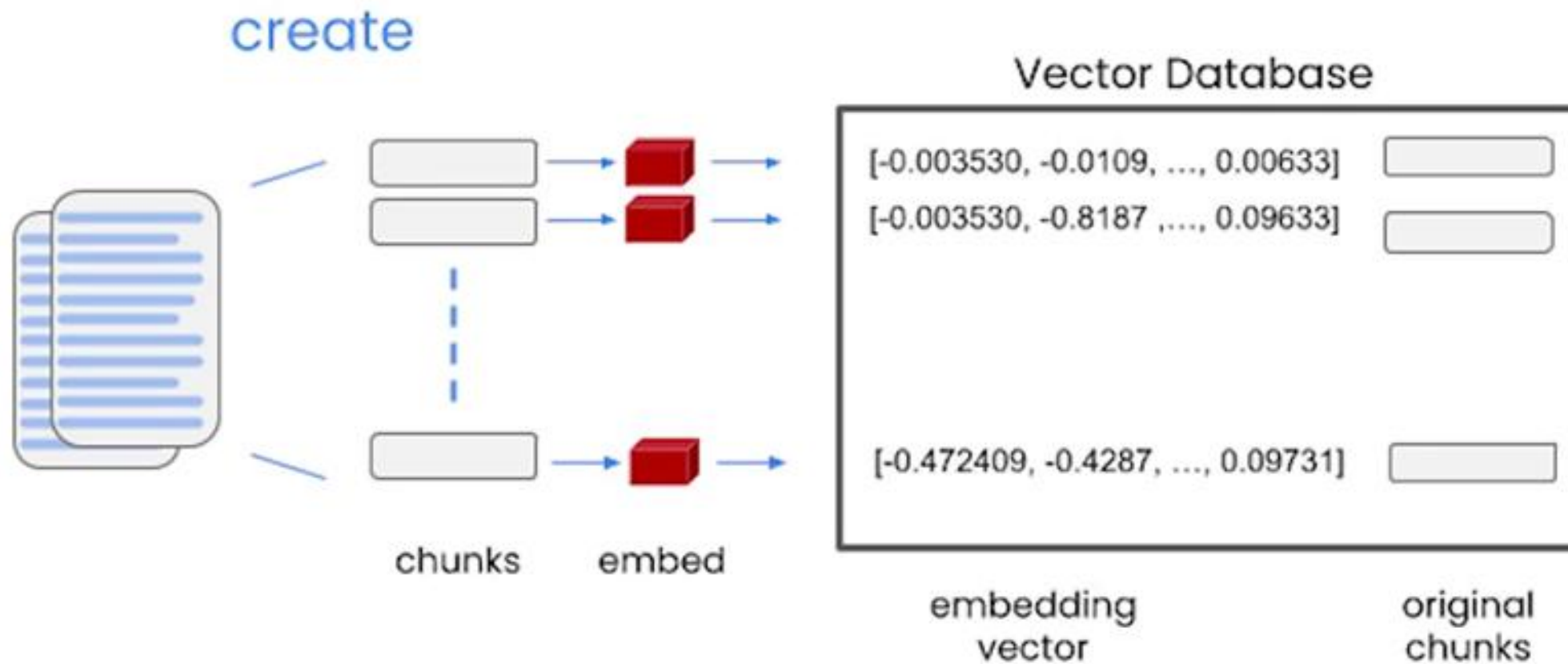


Vector Stores

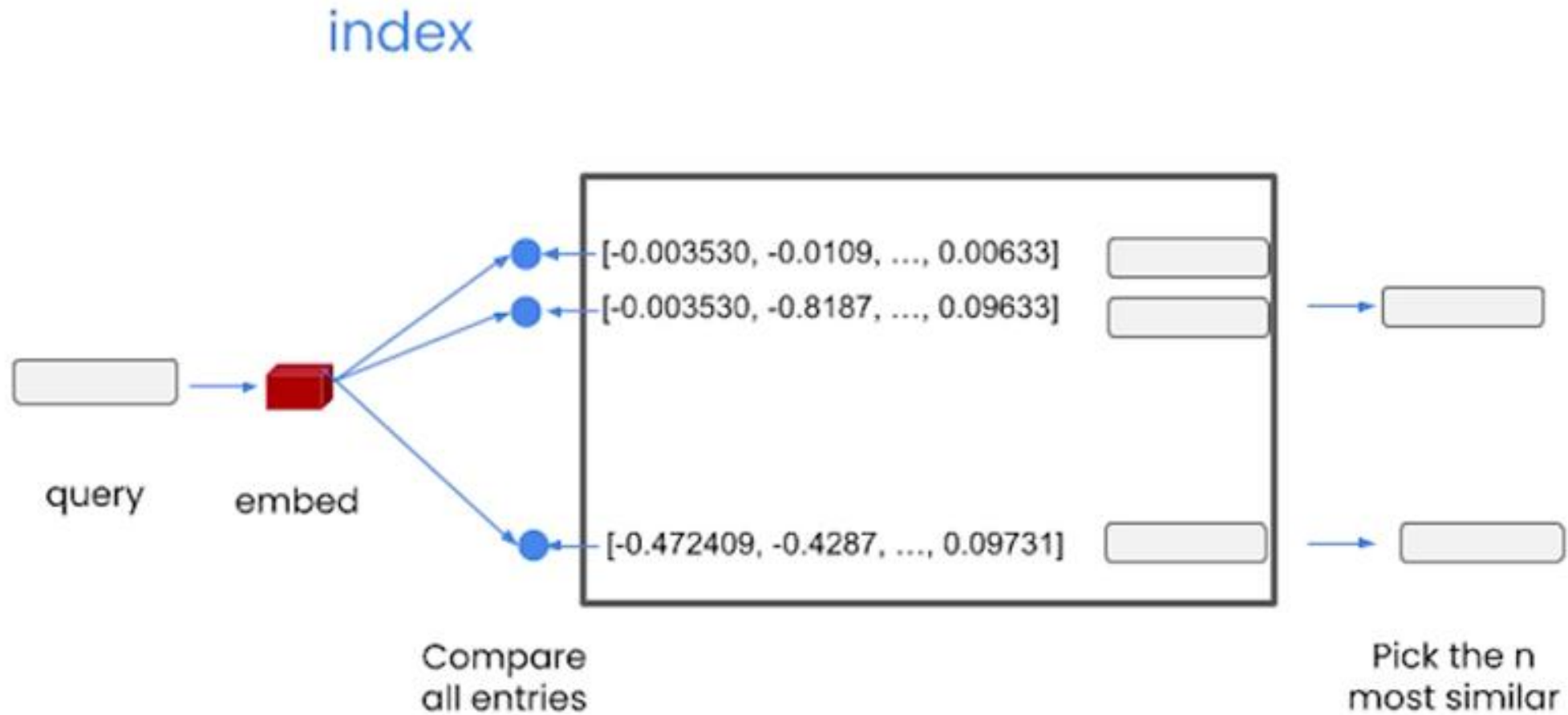


- One of the most common ways to store and search over unstructured data is to embed it and store the resulting embedding vectors, and then at query time to embed the unstructured query and retrieve the embedding vectors that are 'most similar' to the embedded query.
- A vector store takes care of storing embedded data and performing vector search for you.

Vector Store



Vector Store



Vector Store



Process with llm



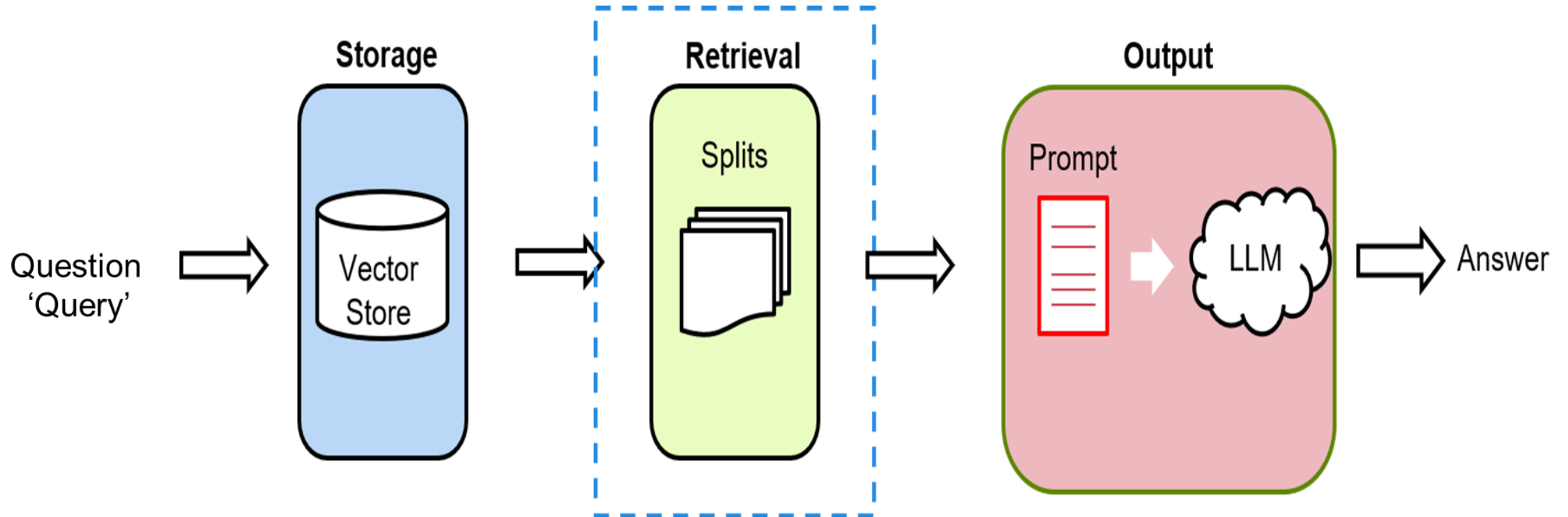
The returned values can now fit in the LLM context

Demo



- Retrieval refers to the process of accessing and fetching relevant pieces of information from a collection of documents or data sources based on a query.
- This involves using techniques like similarity search and embedding-based methods to find and return the most pertinent documents or text segments that match the query's context and content.

Retrieval





- Accessing/indexing the data in the vector store
 - Basic semantic similarity
 - Maximum marginal relevance
 - Including Metadata
- LLM Aided Retrieval

Why Do We Need Retrieval?



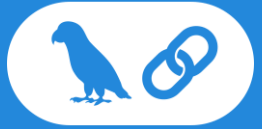
- Enhanced Information Access
- Improved Response Quality
- Efficiency and Performance
- Flexibility and Customization
- Support for Complex Queries
- Dynamic Knowledge Updating

Types of Retrieval



- Vectorstore Retrieval
- Similarity Search
- Maximum marginal Relevance
- Metadata Filtering
- Metadata using Self-Query Retriever

Maximum Marginal Relevance(MMR)

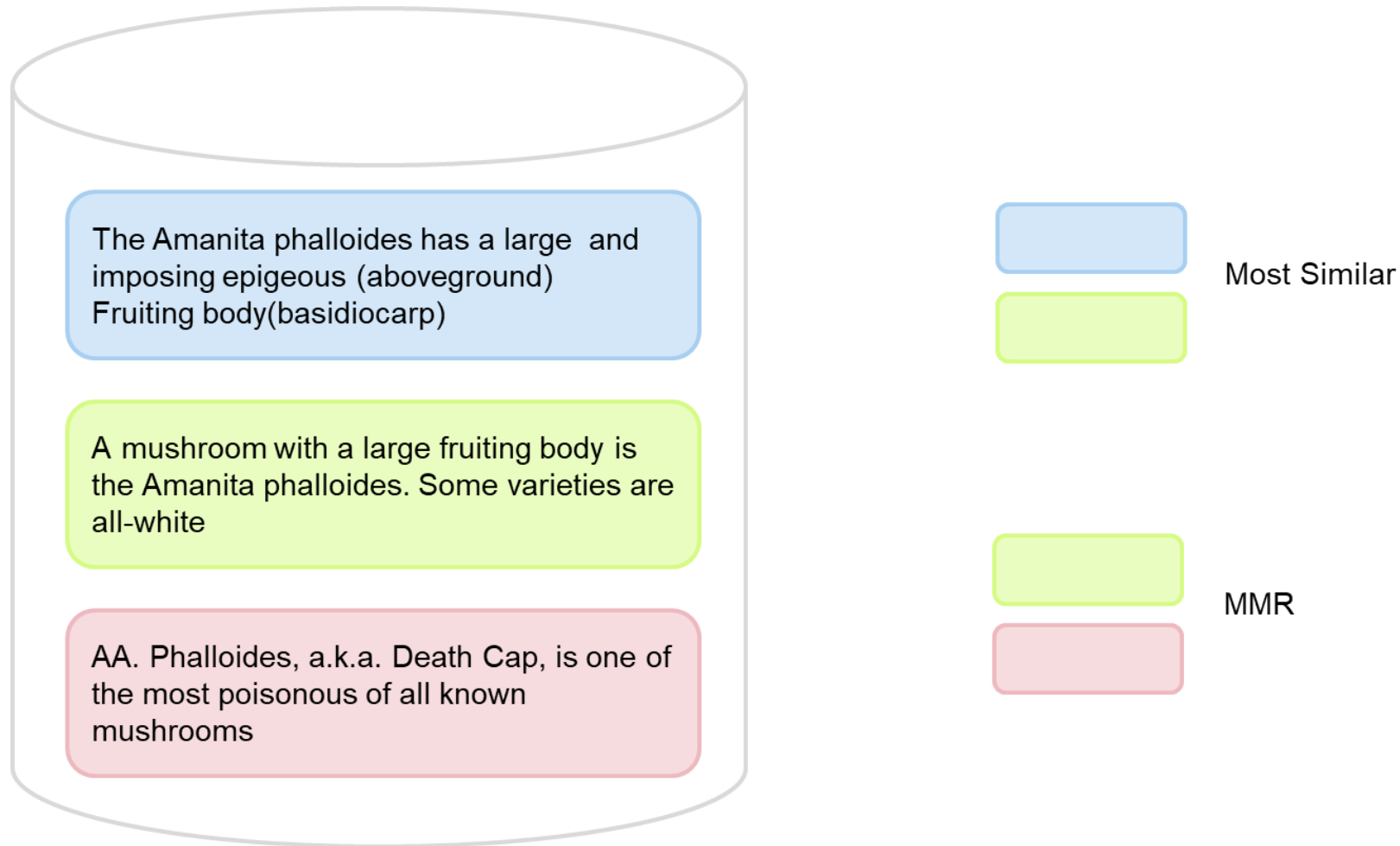
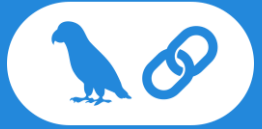


- You may not always want to choose the most similar responses



Tell me about all-white mushrooms with
large fruiting bodies

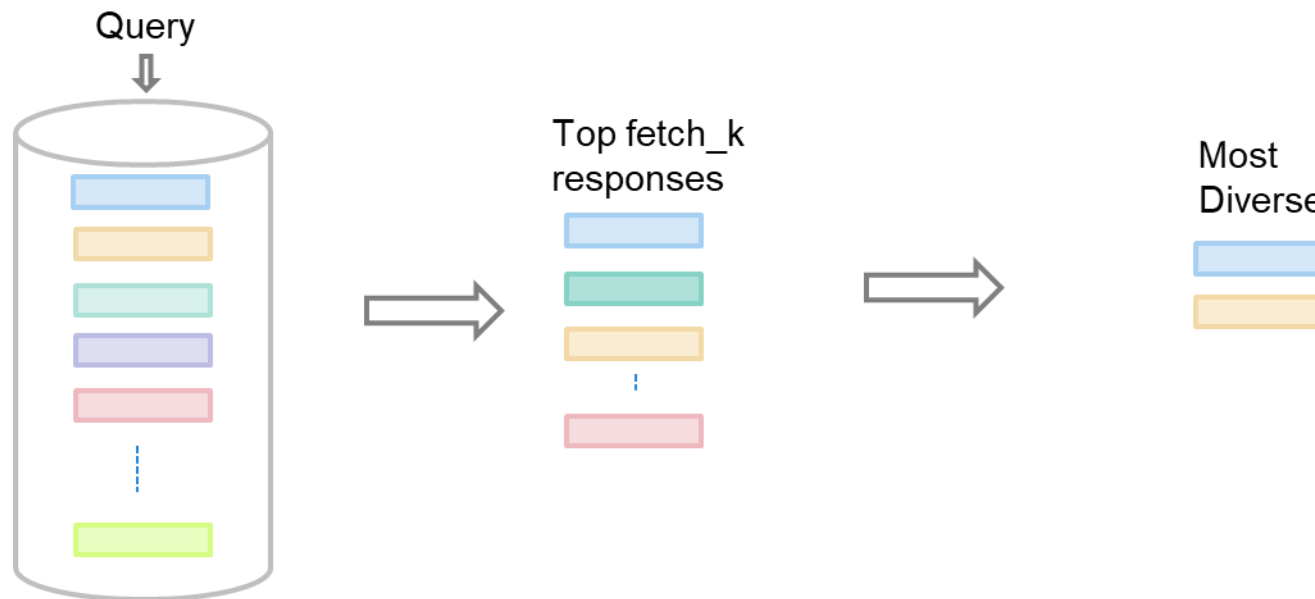
Maximum Marginal Relevance(MMR)



MMR algorithm



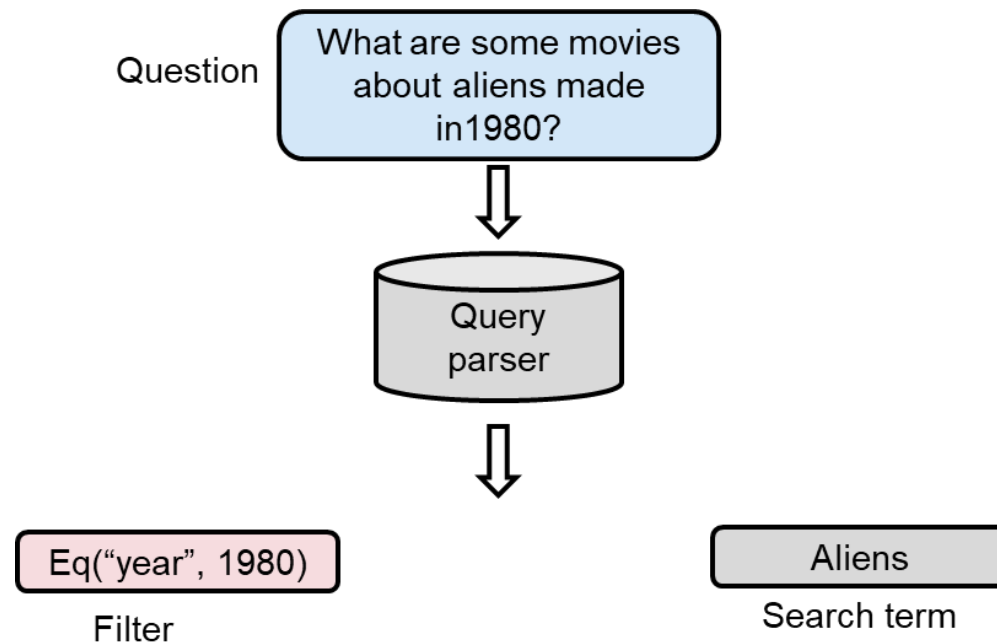
- Query the Vector Store
- Choose the “fetch_kTM” most similar responses
- Within those responses choose the 'k' most diverse



LLM Aided Retrieval



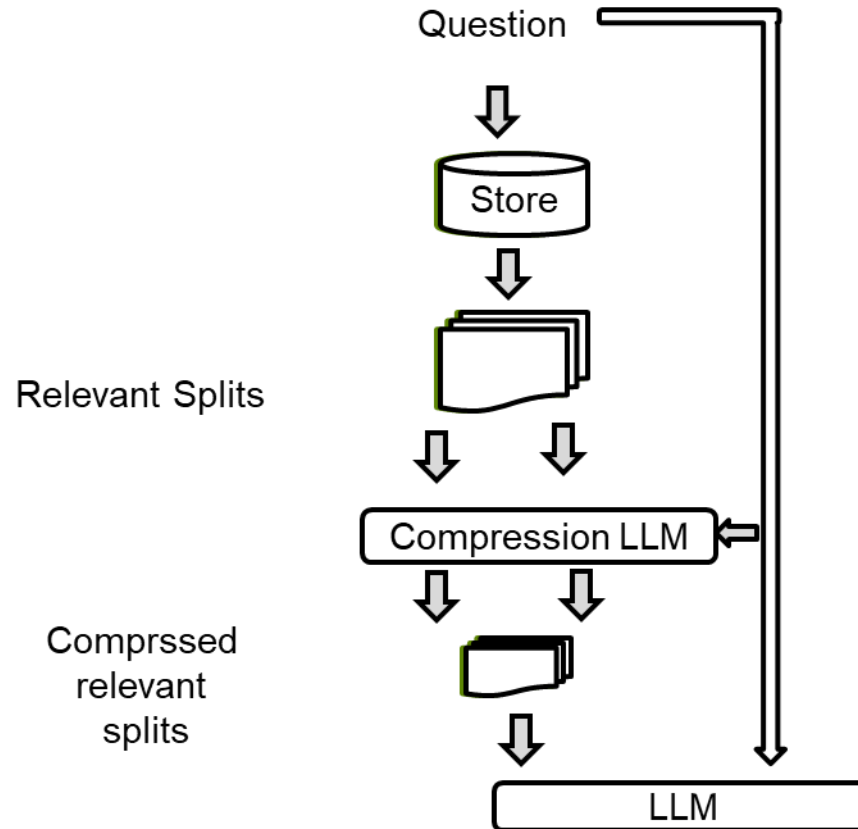
- There are several situations where the applied to the DB is more than just the Question asked.
- One is Self Query, where we use an LLM to convert the user question into a query



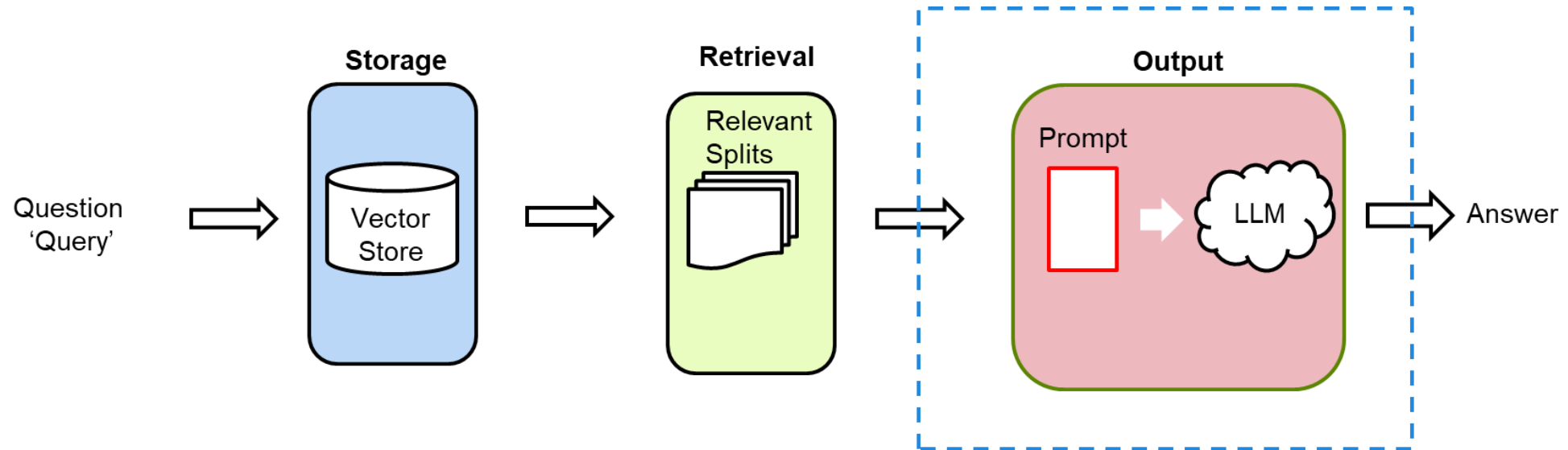
Compression



- Increase the number of results you can put in the context by shrinking the responses to only the relevant information.



Question Answering



- Multiple relevant documents have been retrieved from the vector store
- Potentially compress the relevant splits to fit into the LLM context
- Send the information along with our question to an LLM to select and format an answer

Thank You